

*

Numpy and array notes



1. What is ~~python~~ ^{Numpy}?

Numpy stands for numerical python.

Numpy is the python library used to work with number, arrays, and mathematical calculations.

eg:-

```
import numpy as np
```

```
a = np.array([10, 20, 30, 40])
```

```
print(a)
```

output:

```
[10, 20, 30, 40]
```

2. What is an array.

An array is a collection of elements stored together.

eg:-

[10, 20, 30, 40] → Here, 10, 20, 30, 40

are the elements of the array.

3. Advantage of Numpy

- It is faster for numerical calculations.

- It provides simple functions for calculation.

- Uses less memory - Numpy arrays are generally more memory-efficient.

- Work with large data.

- Support multiple dimensions - we can create 1D, 2D, 3D arrays etc.

- Useful for data analysis - Numpy is commonly used with pandas and other data.

4. Disadvantage

- = use only one data type
- = Not suitable for all types of data
- = Need extra installation.
- = ~~we can~~ use more memory for some operation $\therefore$ large array can require a lot of memory.
- = Difficult for beginners $\therefore$ some numpy function and multidimensional array can be confusing at first.
- = For a small amount of data, a normal python list can sometime be easier.

5. Uses of numpy

numpy is used for

- 1) numerical calculation
- 2) Data analysis
- 3) working with array
- 4) Mathematical operations
- 5) Machine Learning
- 6) Scientific calculation
- 7) Handling large numerical data.

6. Different Between python list and numpy array.

python list	numpy array
- Built in python	comes from numpy library
- can store different datatype	usually store some datatype
- use more memory	generally use less memory
- good for general data	good for numerical data

eg:- Python list

```
a = [10, 20, 30, 40]
```

numpy array

```
import numpy as np
```

```
a = np.array([10, 20, 30, 40])
```

In python list is for general-purpose data, while numpy array is especially useful for numerical data.

G. Element of an array

The individual values inside an array are called elements.

eg:-

```
a = np.array([10, 20, 30, 40])
```

Here

10 → First element

20 → Second "

30 → Third "

40 → Fourth "

So the array has 4 elements.

7. Homogeneous nature of numpy array.

Homogeneous means an array normally stores elements of the same data type.

```
a = np.array([10, 20, 30])
```

All elements are integers.

another example

```
a = np.array([1.2, 2.3, 3.8])
```

All elements are float.

example

```
a = np.array([10, 20, 30])  
print(a.dtype)
```

output

int64

• numpy array are generally homogenous meaning they store the same data

8. Dimensions of array.

Dimension tell us how many levels an array has.

1D array.

one row or a simple list of values.

```
a = np.array([10, 20, 30, 40])
```

This is a 1-dimensional array

2D array

rows and columns.

```
a = np.array([  
    [10, 20],  
    [30, 40]  
])
```

This is a 2-dimensional array.

3D array

It contains multiple 2D arrays.

```
a = np.array([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]]
])
```

This is 3-dimensional array.

9. Shape of an array

The shape tells us the number of elements in each dimension.

we use,

```
array.shape
```

eg:-

```
a = np.array([
    [10, 20, 30],
    [40, 50, 60]
])
```

```
print(a.shape)
```

output

(2, 3) → This means 2 rows and 3 columns

10. ~~str~~ Array data type.

The data type tells us what type of data is stored in the array.

```
array.dtype
```

eg:-

```
import numpy as np
a = np.array([10, 20, 30])
print(a.dtype)
```

output

int64

Some common numpy data type are:-

int → Integer

float → decimal number

bool → true/false

str → text

11. Array Indexing

Indexing means accessing a particular element for an array.

python indexing start from 0.

eg:-

```
a = np.array([10, 20, 30, 40, 50])
```

elements	positive index	negative index
10	0	-5
20	1	-4
30	2	-3
40	3	-2
50	4	-1

positive indexing

positive indexing starts from 0 from the left side.

```
a = np.array([10, 20, 30, 40, 50])
```

```
print(a[0])
```

```
print(a[2])
```

output

10

30

Negative Indexing

```
a = np.array([10, 20, 30, 40, 50])
print(a[-1])
print(a[-2])
```

output

50

40

left $\rightarrow 0, 1, 2, 3, \dots$

right $\rightarrow -1, -2, -3, \dots$

12. Array slicing

Slicing means taking a part of an array.

array[start : stop : step]

start $\rightarrow$ where to start

stop $\rightarrow$ where to stop

step $\rightarrow$ how many position to jump

eg:-

```
a = np.array([10, 20, 30, 40, 50])
print(a[1:4])
```

output

[20, 30, 40] $\rightarrow$ because index 1, 2, 3 are selected

using step

```
a = np.array([10, 20, 30, 40])
print(a[0:5:2])
```

output

[10, 30]

Here 2 mean jump by 2 position.

18. multiple array / multidimensional array.
when an array contain more than one row or dimension, we call it a multidimensional array.

eg:

2D Array eg:

```
import numpy as np
a = np.array([
    [10, 20, 30],
    [40, 50, 60]
])
```

print(a)

output

```
[[10, 20, 30]
 [40, 50, 60]]
```

It has

2 rows

3 column

6 elements

2 dimensions.

* most important syntax to remember:

```
import numpy as np
```

```
a = np.array([10, 20, 30, 40, 50])
```

a[0] * positive indexing

a[-1] * negative "

a[1:4] * slicing

a.shape * shape

a.dtype * data type.